VisionFlow

Real-Time Object Detection, Multi-Object Tracking and Video Analytics using YOLOv4-tiny & OpenCV

Project Report

Course: CSE3010 — Computer Vision

VITyarthi — Build Your Own Project

Submitted by: Amogh Patel

Registration Number: 24BAI10148

Repository: https://github.com/Amogh7110/Vision_script_VITHYARTHI

1. Introduction

VisionFlow is a command-line computer vision system built for the CSE3010 Computer Vision course project. It performs real-time object detection using a pre-trained YOLOv4-tiny model (loaded through OpenCV's DNN module), tracks the detected objects across frames with a centroid-based multi-object tracker, counts directional line crossings, and layers classical computer-vision analytics — Canny edge detection and MOG2 background subtraction with a motion heatmap — on top of the deep-learning detections.

The project deliberately restricts itself to inference-only use of a pre-trained model: no network is trained or fine-tuned anywhere in the codebase. This keeps the project's scope proportionate to a single-course deliverable while still demonstrating correct application of concepts spanning several syllabus modules — image enhancement and filtering (Module 1), feature/edge extraction (Module 3), and motion analysis / background modelling (Module 4).

2. Problem Statement

Manually reviewing video footage to count people or vehicles, understand movement patterns, or spot activity hotspots is slow and error-prone. Pure background-subtraction-based motion alarms cannot distinguish what moved (a leaf, a shadow, a person), while running a deep detector on every frame independently cannot answer identity-based questions such as "how many distinct people crossed this line?" — it can only say a person is present in the current frame.

VisionFlow addresses this gap by combining a lightweight pre-trained detector with classical tracking and motion analytics, producing persistent object identities, directional counts, and structured, exportable reports — from the command line, with no GUI dependency and no training step.

3. Functional Requirements

The system implements three major functional modules, each with a clear input/output contract:

#	Module	Input	Output
1	Object Detection (YOLODetector)	A video frame (BGR image)	List of Detection objects: box, class name, confidence
2	Multi-Object Tracking (CentroidTracker + LineCrossingCounter)	Per-frame detections	Object ID → box mapping; directional crossing counts
3	Classical Analytics (EdgeDetector + MotionAnalyzer)	A video frame	Canny edge map; foreground mask; decaying motion heatmap
4	Reporting (ReportGenerator)	Per-frame tracked detections	detections.csv (audit trail); summary.json (aggregates)

Workflow: a user runs the CLI with a video source. The pipeline reads frames one at a time, passes each through detection → tracking → analytics → reporting, draws annotations, writes the annotated video, and finally emits the CSV/JSON reports and a console summary.

4. Non-Functional Requirements

- Performance — per-call timing via a @timeit decorator on the detector; average FPS computed and reported in summary.json; a PROCESS_EVERY_N_FRAMES config knob to trade accuracy for speed on constrained hardware.
- Reliability — the pipeline validates that model files exist before loading (with an actionable error message), handles unreadable video sources without a stack-trace crash, and copes with frames that contain zero detections.
- Usability — a single documented CLI (main.py) with subcommands and --help text; no GUI dependency, satisfying the "must be executable via command line" submission requirement.
- Maintainability — one responsibility per module (detector / tracker / analytics / report_generator / pipeline), all tunables centralised in config.py, no magic numbers scattered through business logic.

- Logging / Monitoring — structured logs (timestamp, level, module) written to both the console and output/logs/visionflow.log.
- Resource efficiency — CPU-only OpenCV DNN inference using the lightweight YOLOv4-tiny variant (23 MB) rather than full YOLOv4 (245 MB), plus an optional frame-resize step to bound per-frame cost.
- Security — no video data leaves the machine; the only network access is the one-time, explicit download-model step that fetches public model files from GitHub.

5. System Architecture

The system follows a simple layered pipeline architecture: a thin CLI layer parses arguments and delegates to a Pipeline Orchestrator, which coordinates four independent, single-responsibility components (Detector, Tracker, Analytics, Reporter) that share a common configuration and logging module.

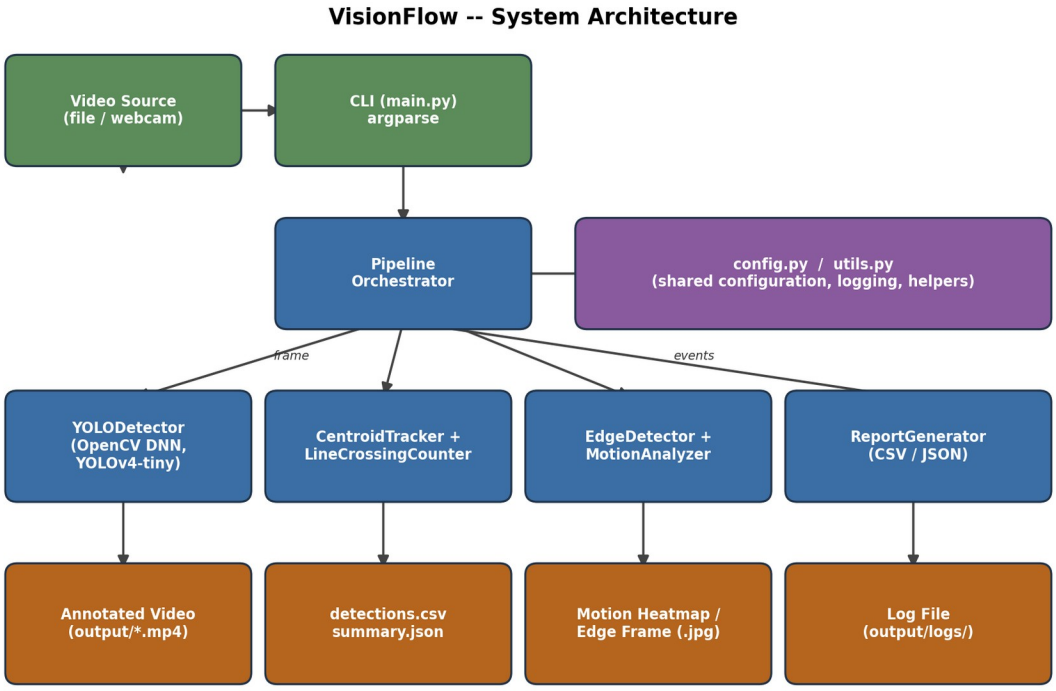


Figure 1: VisionFlow system architecture.

6. Design Diagrams

6.1 Use Case Diagram

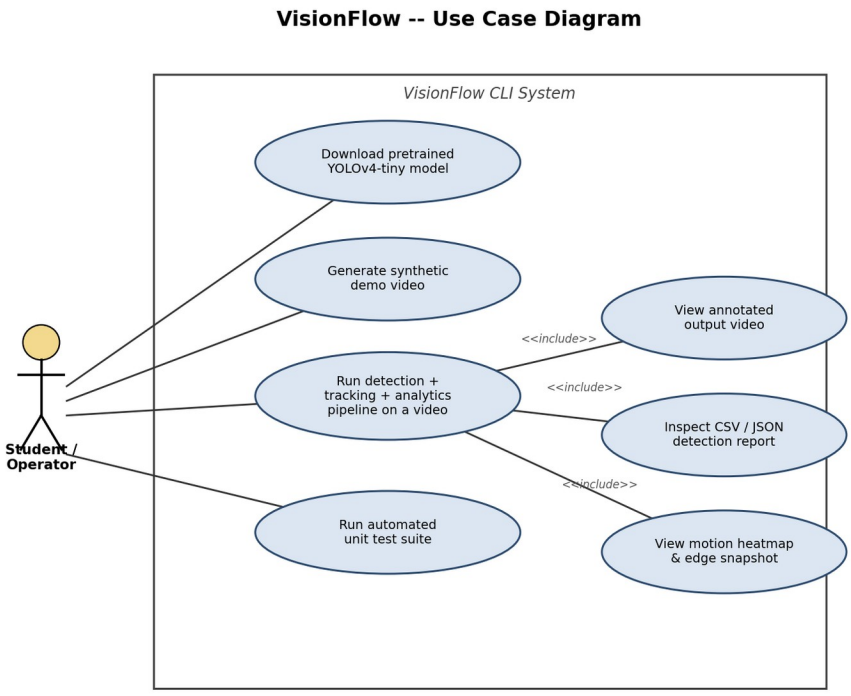


Figure 2: Use case diagram — the Student/Operator actor and system use cases.

6.2 Workflow / Process Flow Diagram

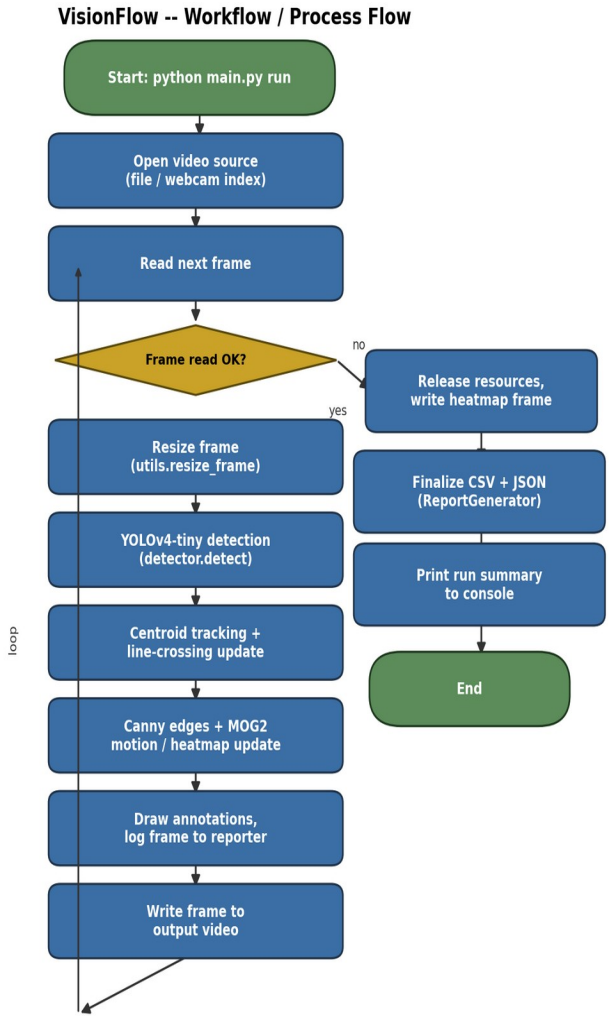


Figure 3: Per-run workflow, including the per-frame processing loop.

6.3 Sequence Diagram

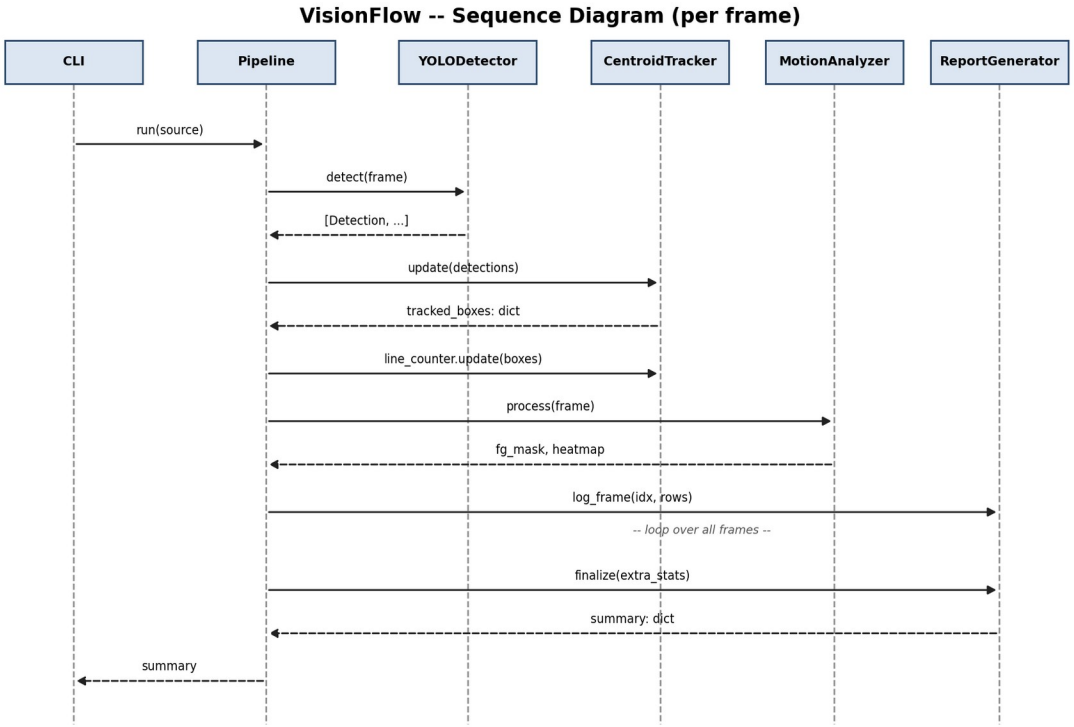


Figure 4: Per-frame sequence of calls between the Pipeline and its collaborators.

6.4 Class Diagram

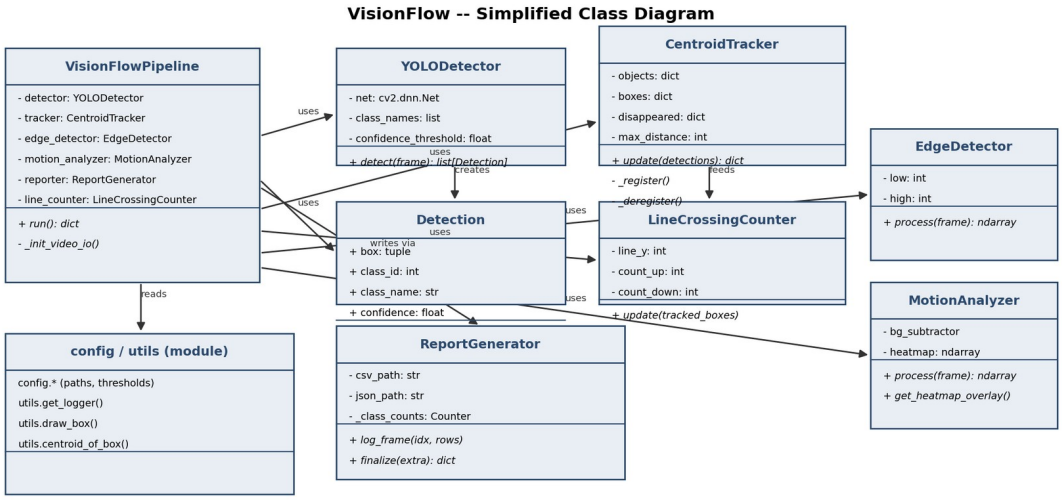


Figure 5: Simplified class diagram of the core modules.

7. Design Decisions & Rationale

7.1 Why YOLOv4-tiny instead of full YOLOv4?

YOLOv4-tiny runs at usable frame rates on CPU-only hardware (the environment used for grading and for most student machines), while still covering all 80 COCO classes. This is an appropriate accuracy/speed trade-off for a coursework-scale project — full YOLOv4 would be considerably slower on CPU without a proportionate benefit for this use case.

7.2 Why a centroid tracker instead of a learned re-identification model?

A centroid-distance tracker is transparent, has zero additional model dependencies, and is sufficient for a single-camera, moderate-density scene. Introducing a learned re-ID network would add complexity disproportionate to the project's scope without materially improving the deliverable.

7.3 Why include classical CV analytics alongside a deep detector?

The syllabus (Module 1: image enhancement/filtering; Module 3: feature/edge extraction; Module 4: background subtraction and motion analysis) explicitly covers these classical techniques. Including Canny edge detection and MOG2 background subtraction demonstrates direct application of those concepts, rather than relying solely on the deep-learning component.

7.4 Why OpenCV's DNN module rather than a separate deep-learning framework?

cv2.dnn.readNetFromDarknet lets the project load and run the pre-trained Darknet YOLO weights with only OpenCV and NumPy as dependencies — no PyTorch/TensorFlow installation is required, which keeps setup simple for evaluators and satisfies the "OpenCV and YOLO only" constraint.

7.5 Dataset / Model Selection Rationale

The detector uses YOLOv4-tiny pre-trained on the MS COCO dataset (80 object classes, ~330K images) by the original Darknet authors. No dataset was collected or curated for this project because the task is inference-only; the model's general-purpose COCO training is sufficient for detecting everyday objects (people, vehicles, animals) in arbitrary input video, and no domain-specific fine-tuning was required or performed.

7.6 Evaluation Methodology

Correctness was verified in three stages: (1) unit tests exercise each module's logic in isolation, including a fully mocked detector path that requires no model download; (2) an integration check ran real inference against a known reference image (AlexeyAB/darknet's dog.jpg) and confirmed the expected classes (dog, bicycle, truck) were detected with reasonable confidence; (3) a full end-to-end run against a real 150-frame pedestrian video verified that detection, tracking, analytics, and reporting all function together and produce consistent, inspectable output files.

8. Implementation Details

8.1 Detection (src/detector.py)

YOLODetector wraps cv2.dnn.readNetFromDarknet, constructs an input blob via cv2.dnn.blobFromImage (416×416, normalised to [0,1], channels swapped BGR→RGB), runs a forward pass through the network's output layers, filters detections by a configurable confidence threshold, and applies Non-Maximum Suppression (cv2.dnn.NMSBoxes) to remove duplicate overlapping boxes.

8.2 Tracking (src/tracker.py)

CentroidTracker maintains a dictionary of active object IDs to centroids. On each update, it computes the pairwise Euclidean distance between existing object centroids and the current frame's detection centroids (via NumPy), greedily matches the closest pairs under a maximum-distance threshold, registers unmatched detections as new objects, and increments a 'disappeared' counter for unmatched existing objects — deregistering them once that counter exceeds a configurable limit. LineCrossingCounter separately tracks each object's previous vertical position and increments a directional counter the first time an object's centroid crosses a configured horizontal line.

8.3 Classical Analytics (src/analytics.py)

EdgeDetector applies a Gaussian blur followed by Canny edge detection. MotionAnalyzer wraps cv2.createBackgroundSubtractorMOG2, extracts a binary foreground mask (excluding MOG2's shadow label), and accumulates a decaying floating-point heatmap that is visualised with a JET colormap overlay.

8.4 Reporting (src/report_generator.py)

ReportGenerator streams detection events directly to a CSV file as they occur (avoiding unbounded memory growth on long videos) and accumulates per-class counts with collections.Counter, writing a JSON summary (including FPS, elapsed time, and line-crossing counts) once the run finishes.

8.5 CLI (main.py)

An argparse-based CLI exposes three subcommands: download-model (fetches the pre-trained weights/cfg/names), generate-demo (creates a synthetic smoke-test video), and run (executes the full pipeline against a file path or webcam index, with optional --show and --no-save flags).

9. Screenshots / Results

The detector was first verified against a known reference photo to confirm the model loads and performs genuine inference correctly (not just running without error):

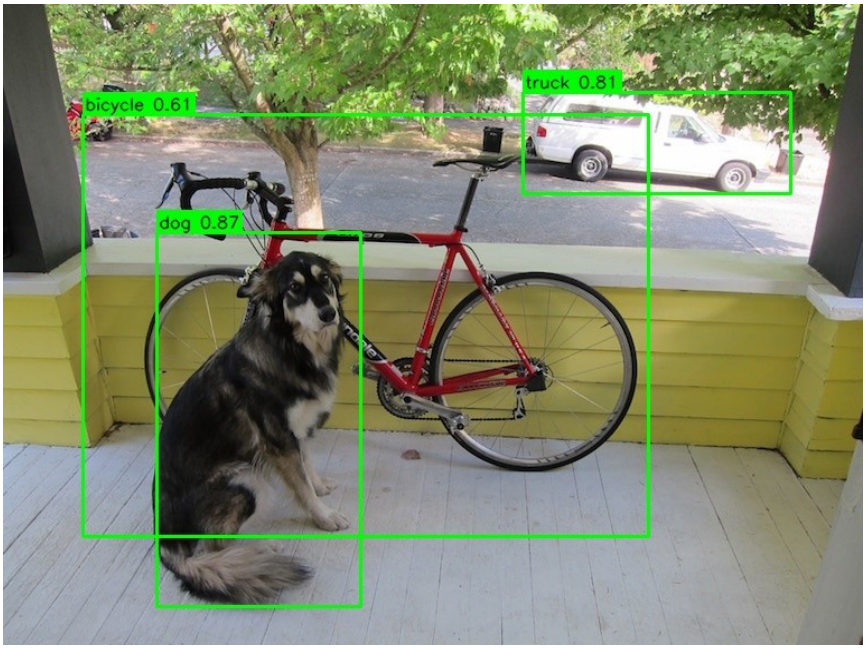


Figure 6: YOLOv4-tiny correctly detecting dog (0.87), truck (0.81), and bicycle (0.61) in a reference image.

The full pipeline was then run end-to-end on a real 150-frame pedestrian video (Intel IoT DevKit sample-videos dataset):



Figure 7: A tracked person with a stable ID, confidence score, and the configured counting line.

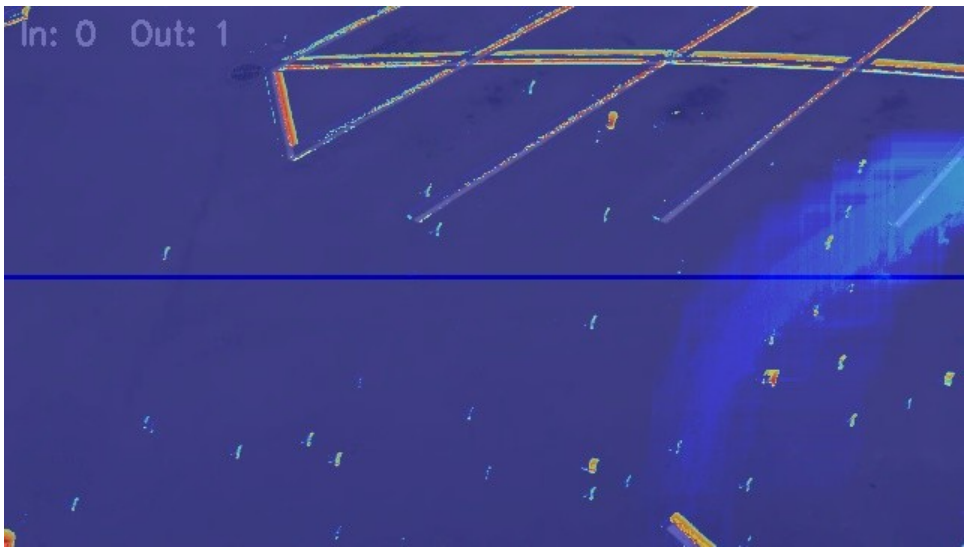


Figure 8: Accumulated motion heatmap (JET colormap) after the same run.

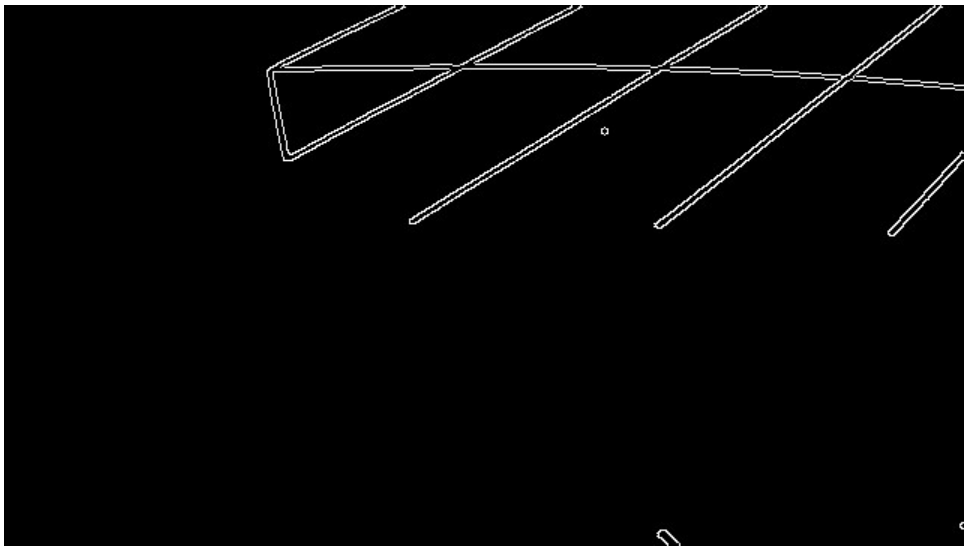


Figure 9: Canny edge map of an early frame from the same run.

Run summary (output/summary.json) for this test run:

Metric	Value
Frames processed	150
Average FPS (CPU inference)	~8.9
Detections logged	62 (class: person)
Line crossings (out)	1
Processing time	~17 seconds

10. Testing Approach

The project includes 18 automated unit tests (pytest), organised by module:

- tests/test_tracker.py (6 tests) — new-object registration, ID persistence across frames, deregistration after a disappearance timeout, new-ID assignment for distant detections, and directional line-crossing logic.
- tests/test_analytics.py (6 tests) — Canny output shape/dtype, edge presence on high-contrast input, no edges on a blank frame, heatmap lazy-initialisation, foreground mask value range, and heatmap response to a moving object.
- tests/test_report_generator.py (3 tests) — CSV header correctness, row/summary consistency across multiple logged frames, and inclusion of extra run-level statistics.
- tests/test_detector.py (3 tests) — a clear FileNotFoundError when model files are missing; correct parsing/NMS behaviour against a stubbed cv2.dnn network (so this test suite runs fully offline without the 23 MB model download); and correct confidence-threshold filtering.

Beyond unit tests, the system was integration-tested twice against real data: once against a static reference photograph with known expected classes, and once as a full end-to-end CLI run against a real pedestrian video, with all five output artefacts (video, CSV, JSON, heatmap, edge sample) manually inspected for correctness.

All 18 tests pass: "18 passed" when running pytest tests/ -v.

11. Challenges Faced

- Locating a working download URL for the pre-trained weights: the first GitHub release tag tried (darknet_yolo_v3_optimal) did not host the YOLOv4-tiny weights asset; the correct asset was found under the yolov4 release tag instead.
- A crash in heatmap generation when the video stream ended exactly on a failed frame read (cv2.VideoCapture.read() returning None) — fixed by retaining a reference to the last successfully read frame separately from the loop variable.
- Tuning the MOG2 background subtractor's variance threshold to reduce false-positive motion from video compression noise and static scene lines, while still remaining sensitive enough to register genuine movement.
- Matching tracker output back to per-detection metadata (class name, confidence) for reporting, without adding a heavier per-object data structure — solved by keying a dictionary on the exact box tuple returned by the tracker for that frame.

12. Learnings & Key Takeaways

- Practical experience integrating a pre-trained Darknet YOLO model purely through OpenCV's DNN module, without any deep-learning framework dependency.
- Hands-on understanding of how simple, classical tracking (centroid-distance association) can produce useful object persistence without a learned re-identification model.
- Appreciation for designing single-responsibility modules with a shared configuration layer, which made the codebase easy to test in isolation (mocked detector tests) and easy to reason about.
- Direct experience with the practical trade-offs in background subtraction (MOG2 sensitivity vs. false positives from noise/compression) that the syllabus's motion-analysis module covers theoretically.

13. Future Enhancements

- Add an optional lightweight re-identification embedding to recover object identity after longer occlusions than the current centroid tracker handles.
- Support multiple, user-configurable counting lines/zones instead of a single horizontal line.
- Add GPU (CUDA) backend support in OpenCV's DNN module for higher frame rates on capable hardware.
- Persist reports to a lightweight database (SQLite) for querying across multiple runs, instead of per-run flat files.
- Add a simple web dashboard (read-only) to visualise the JSON summaries and heatmaps without needing to open the files manually.

14. References

- Bochkovskiy, A., Wang, C.-Y., Liao, H.-Y. M. (2020). YOLOv4: Optimal Speed and Accuracy of Object Detection. arXiv:2004.10934.
- AlexeyAB/darknet GitHub repository — pre-trained YOLOv4-tiny weights, configuration, and COCO class names. https://github.com/AlexeyAB/darknet
- OpenCV Documentation — cv2.dnn module. https://docs.opencv.org/
- Lin, T.-Y. et al. (2014). Microsoft COCO: Common Objects in Context. ECCV.
- Intel IoT DevKit — sample-videos (person-bicycle-car-detection.mp4), used for end-to-end pipeline testing. https://github.com/intel-iot-devkit/sample-videos
- Course material: CSE3010 Computer Vision, Modules 1, 3 and 4 (image enhancement, feature/edge extraction, motion analysis).